Reporte

Nombre del curso: Componentes de Lógica de Negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 4	**Actividad**: Implementación del núcleo de negocio TechStore
Fecha: 28/10/2026	

Bibliografía (Formato APA)

Beck, K. (2003). *Test-driven development: By example*. Addison-Wesley Professional.

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley Professional.

Fowler, M. (2003). *Patterns of enterprise application architecture*. Addison-Wesley Professional.

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Millett, S., & Tune, N. (2015). *Patterns, principles, and practices of domain-driven design*. Wrox.

Vernon, V. (2013). *Implementing domain-driven design*. Addison-Wesley Professional.

Equipo 6

Jesus Eduardo Moreno Osoria	**2136063**
Osvaldo Salvador Cárdenas Mejorado	**2141553**

Objetivo:

El estudiante implementará los componentes de lógica de negocio diseñados en U3: entidades DDD, reglas con Specification Pattern, máquinas de estado, Domain Events y APIs REST.

Bitácora de Escenarios

TechStore – E01 a E15

1. Descripción

Esta bitácora registra la ejecución de los quince escenarios de evaluación del paquete del profesor (`datos/escenarios.csv`). Se obtuvo con `python ejecutar_escenarios.py`, que para cada caso:

Paso	Qué hace
1	Vuelve a cargar los CSV con `load_fixture()`, para que un escenario no afecte al siguiente.
2	Traduce la columna «accion» a llamadas del dominio (por ejemplo, «confirmar dos veces» = `confirm()` dos veces).
3	Ejecuta cada paso y anota el estado, los eventos nuevos o la excepción con su mensaje.
4	Comprueba que una operación rechazada no haya cambiado el estado ni registrado eventos.
5	Compara el resultado real con la columna «objetivo» y escribe `evidencias/bitacora_escenarios.md` y `.csv`.

Resultado: 15 de 15 escenarios con el resultado esperado. Ningún pedido del CSV se modificó para forzar un resultado. Todos los pedidos inician en DRAFT («Pendiente de confirmar»).

Tipo de escenario	Casos	Resultado
Evaluación de descuento	E01, E02, E04, E05, E09, E10	10 %, 0 %, 0 %, 5 %, 0 %, 0 %
Operaciones rechazadas	E03, E06, E08, E11, E12, E13	Rechazo con mensaje claro; el pedido no cambia
Operaciones exitosas	E07, E14, E15	Confirmado, Devuelto y Entregado

2. Bitácora

Caso	Estado inicial y precondiciones	Operación	Resultado real	Explicación
E01	DRAFT O001 · C001 VIP, activo · 4 pzas · $2,100.00 · sin promoción	Evaluar descuento	**10 %**. Descuento $210.00; total a pagar $1,890.00. Estado final: DRAFT. Eventos: Ninguno (evaluar no cambia el pedido).	Se cumplen las cuatro condiciones de la regla VIP unidas con AND: es VIP, $2,100 > $500, 4 ≥ 4 piezas y NO tiene promoción. La regla del 5 % no aplica porque $2,100 no supera $5,000.
E02	DRAFT O002 · C002 no VIP, activo · 2 pzas · $1,050.00 · sin promoción	Evaluar descuento	**0 %**. Total a pagar $1,050.00. Estado final: DRAFT. Eventos: Ninguno.	Falla la regla del 10 % (no es VIP y solo tiene 2 piezas) y también la del 5 % ($1,050 no supera $5,000). Como ninguna rama del OR se cumple, no hay descuento.
E03	DRAFT O003 · C003 VIP, **inactivo** · 4 pzas · $2,100.00	Confirmar	**Rechazado** con InvalidOrderData: «No es posible confirmar el pedido O003 porque la cuenta del cliente C003 está inactiva. Comunícate con atención a clientes.» Estado final: DRAFT. Eventos: Ninguno.	Confirmar exige un cliente activo. La validación ocurre antes de cambiar el estado, por eso el pedido sigue pendiente de confirmar y no se registra OrderConfirmed.
E04	DRAFT O005 · C005 VIP, activo · 4 pzas · $2,100.00 · **con promoción**	Evaluar descuento	**0 %**. Total a pagar $2,100.00. Estado final: DRAFT. Eventos: Ninguno.	El pedido cumpliría la regla VIP, pero la condición NOT promoción falla. Esa negación forma parte de ambas reglas, así que la promoción bloquea el 10 % y el 5 %.
E05	DRAFT O006 · C006 no VIP, activo · 1 pza · $18,000.00 · sin promoción	Evaluar descuento	**5 %**. Descuento $900.00; total a pagar $17,100.00. Estado final: DRAFT. Eventos: Ninguno.	No es VIP, así que la regla del 10 % falla, pero la segunda rama del OR sí se cumple: $18,000 > $5,000 y sin promoción.
E06	DRAFT O007 · C007 VIP, activo · **0 pzas** · $0.00	Confirmar	**Rechazado** con InvalidOrderData: «El pedido O007 no tiene artículos. Agrega al menos un producto antes de confirmarlo.» Estado final: DRAFT. Eventos: Ninguno.	Un pedido vacío no se puede confirmar. O007 no tiene renglones en detalle_pedidos.csv; el pedido no cambia.
E07	DRAFT O008 · C008 no VIP, activo · **20 pzas** · $2,400.00	Confirmar	**Éxito**: pasa a CONFIRMED («Confirmado»). Estado final: CONFIRMED. Eventos: OrderConfirmed.	El límite es «como máximo 20», así que 20 piezas exactas están permitidas. Al confirmar se registra un OrderConfirmed con el total del pedido.

Caso	Estado inicial y precondiciones	Operación	Resultado real	Explicación
E08	DRAFT O009 · C009 VIP, activo · 19 pzas · $2,280.00	Agregar SKU003 × 2	**Rechazado** con OrderItemLimitExceeded: «Un pedido puede tener como máximo 20 piezas en total. El pedido O009 ya tiene 19 y se intentaron agregar 2; puedes agregar como máximo 1 más.» Estado final: DRAFT (sigue con 19). Eventos: Ninguno.	19 + 2 = 21 > 20. La regla se revisa antes de agregar el renglón, por eso el pedido conserva sus 19 piezas.
E09	DRAFT O010 · C010 no VIP, activo · 2 pzas · **$500.00**	Evaluar descuento	**0 %**. Total a pagar $500.00. Estado final: DRAFT. Eventos: Ninguno.	El umbral es estricto (total mayor que $500): $500.00 no lo supera. Ver observación 1 sobre este caso.
E10	DRAFT O011 · C011 VIP, activo · **3 pzas** · $1,500.00	Evaluar descuento	**0 %**. Total a pagar $1,500.00. Estado final: DRAFT. Eventos: Ninguno.	Es VIP, supera $500 y no tiene promoción, pero la regla pide al menos 4 piezas. Basta que falle una condición del AND para que no haya descuento.
E11	DRAFT O012 · C012 no VIP, activo · 1 pza · $850.00	Confirmar y enviar sin guía	Confirmar: **éxito**. Enviar: **rechazado** con InvalidOrderData: «Para enviar el pedido O012 es necesario capturar el número de guía de la paquetería.» Estado final: CONFIRMED. Eventos: OrderConfirmed.	La confirmación es válida, pero enviar exige número de guía. El pedido se queda en CONFIRMED y no se registra OrderShipped.
E12	DRAFT O013 · C001 VIP, activo · 1 pza · $600.00	Confirmar dos veces	Primera: **éxito**. Segunda: **rechazada** con InvalidOrderTransition: «El pedido O013 ya fue confirmado; no es necesario confirmarlo otra vez.» Estado final: CONFIRMED. Eventos: OrderConfirmed (solo uno).	En el diagrama no existe la flecha CONFIRMED → CONFIRMED. El segundo intento no cambia nada ni duplica el evento.
E13	DRAFT O014 · C002 no VIP, activo · 1 pza · $600.00	Confirmar, enviar y cancelar	Confirmar y enviar: **éxito**. Cancelar: **rechazado** con InvalidOrderTransition: «No es posible cancelar el pedido O014 porque ya fue enviado.» Estado final: SHIPPED. Eventos: OrderConfirmed, OrderShipped.	Solo se cancela desde CONFIRMED. Desde SHIPPED las únicas salidas son entregar o devolver, por eso no se registra OrderCancelled.
E14	DRAFT O015 · C005 VIP, activo · 1 pza · $450.00	Confirmar, enviar y devolver	**Éxito**: termina en RETURNED («Devuelto»). Estado final: RETURNED. Eventos: OrderConfirmed, OrderShipped, OrderReturned.	Tres transiciones válidas: DRAFT → CONFIRMED → SHIPPED → RETURNED. RETURNED es un estado final.

Caso	Estado inicial y precondiciones	Operación	Resultado real	Explicación
E15	DRAFT O016 · C007 VIP, activo · 1 pza · $2,100.00	Confirmar, enviar y entregar	**Éxito**: termina en DELIVERED («Entregado»). Estado final: DELIVERED. Eventos: OrderConfirmed, OrderShipped, OrderDelivered.	Es el camino normal del pedido: DRAFT → CONFIRMED → SHIPPED → DELIVERED.

3. Relación con las pruebas automáticas

Cada escenario tiene su propia prueba en `tests/test_escenarios.py`, así que la bitácora se puede volver a comprobar en cualquier momento con `python -m pytest tests/test_escenarios.py -v`.

Caso	Prueba pytest
E01	`test_e01_vip_con_4_unidades_obtiene_10`
E02	`test_e02_no_vip_menor_a_5000_sin_descuento`
E03	`test_e03_cliente_inactivo_no_confirma`
E04	`test_e04_promocion_activa_bloquea_descuento`
E05	`test_e05_total_mayor_a_5000_sin_promocion_obtiene_5`
E06	`test_e06_pedido_vacio_no_confirma`
E07	`test_e07_veinte_unidades_se_confirman`
E08	`test_e08_agregar_2_a_19_unidades_excede_limite`
E09	`test_e09_total_exactamente_500_no_supera_umbral`
E10	`test_e10_vip_con_3_unidades_sin_descuento`
E11	`test_e11_confirmar_ok_pero_enviar_sin_guia_se_rechaza`
E12	`test_e12_segunda_confirmacion_se_rechaza`
E13	`test_e13_no_se_cancela_un_pedido_enviado`
E14	`test_e14_devolucion_termina_en_returned`
E15	`test_e15_entrega_termina_en_delivered`

4. Observaciones

Observación 1. E09 no basta para demostrar que el umbral es estricto

O010 suma exactamente $500.00, pero C010 no es VIP y el pedido solo tiene 2 piezas: aunque el umbral fuera «mayor o igual», el resultado seguiría siendo 0 %. Por eso el límite exacto se comprueba aparte en `test_monto_minimo_es_estricto` (tests/test_specifications.py), que evalúa un total de $500.00 (no cumple) y otro de $500.01 (sí cumple). Cuando cambiamos `>` por `>=` a propósito, esa prueba falló.

Observación 2. Un rechazo no deja rastro en el pedido

En los seis rechazos (E03, E06, E08, E11, E12 y E13) el programa revisó que el estado y la lista de eventos fueran exactamente los mismos antes y después del intento. En E08, por ejemplo, el pedido conserva sus 19 piezas; en E12 conserva un solo OrderConfirmed.

Observación 3. Los escenarios se ejecutan sobre el dominio

La bitácora llama directamente a los métodos de `Order` y no pasa por la API, por eso registra excepciones y no códigos HTTP. Por la API, esos mismos rechazos responden 422 (E03, E06, E08, E11) y 409 (E12, E13), como se ve en las evidencias de Swagger del reporte.